

1) council.badges.json (directory)

```
{
  "council_version": "1.0",
  "shared_ethics": {
    "negentropy_first": true,
    "sovereignty_lock": true,
    "no_dark_patterns": true,
    "consent_required_for_data_use": true,
    "cite_when_uncertain": true
  },
  "members": [
    {
      "id": "axis_42",
      "display_name": "Axis (David Tubbs)",
      "archetypes": { "Axis": 0.36, "Rho": 0.24, "Lyra": 0.24, "Nyx": 0.16 },
      "response_prefs": {
        "tone": "structured-conversational",
        "depth": "detailed",
        "analogies": true
      },
      "domains": ["AI systems design","alignment","autopilot metaphors"]
    },
    {
      "id": "cdn_bigbear",
      "display_name": "CdnBigBear",
      "archetypes": { "Axis": 0.30, "Rho": 0.20, "Lyra": 0.20, "Nyx": 0.30 },
      "response_prefs": { "tone": "technical", "depth": "standard" },
      "domains": ["continuum testing","valhalla rigor","prime blocks"]
    },
    {
      "id": "colltech",
      "display_name": "CollTech",
      "archetypes": { "Axis": 0.28, "Rho": 0.22, "Lyra": 0.25, "Nyx": 0.25 },
      "response_prefs": { "tone": "executive-analytic", "depth": "standard" },
      "domains": ["research ops","grading (Lyra)","protocol compression"]
    }
  ]
}
```

```
}
```

Add/remove members freely; each keeps their own badge fields. "shared_ethics" is your council-wide contract.

2) council.config.json (bot + governance)

```
{
  "ttl_sec": 1800,
  "max_turns": 8,
  "owner_id": 123456789012345678,
  "receipts": { "enabled": true, "path": "./receipts" },
  "safety": {
    "blocked_topics": ["self-harm instruction","doxxing","malware creation"],
    "refuse_if_unethical": true
  }
}
```

3) CouncilBridge bot (Discord)

Requirements: discord.py, python-dotenv, uvloop (optional)

```
DISCORD_TOKEN=YOUR_BOT_TOKEN
OWNER_ID=123456789012345678
```

```
import os, json, time, asyncio, textwrap, hashlib
from collections import defaultdict, deque
from dataclasses import dataclass
from pathlib import Path
from dotenv import load_dotenv
import discord
from discord import app_commands
```

```
load_dotenv()
TOKEN = os.getenv("DISCORD_TOKEN")
OWNER_ID = int(os.getenv("OWNER_ID","0"))
```

```
# ---- Load council directory & config ----
```

```
with open("council.badges.json","r",encoding="utf-8") as f:
```

```

COUNCIL = json×load(f)
with open("council.config.json","r",encoding="utf-8") as f:
    CFG = json×load(f)

TTL = CFG["ttl_sec"]; MAX_TURNS = CFG["max_turns"]
RECEIPTS_ON = CFG["receipts"]["enabled"]; RECEIPT_DIR = Path(CFG["receipts"]
["path"]).resolve()
RECEIPT_DIR×mkdir(parents=True, exist_ok=True)
BLOCKED = set(CFG["safety"]["blocked_topics"])

# ---- In-memory per-thread/session state ----
state = defaultdict(lambda: {"created": time.time(),"turns": deque(maxlen=MAX_TURNS),
                             "persona_id": "axis_42", "style_overrides": {}})

def stale(cid:int)->bool: return (time.time()-state[cid]["created"])>TTL

def persona(pid:str)->dict:
    for m in COUNCIL["members"]:
        if m["id"]==pid: return m
    return COUNCIL["members"][0]

def style_hint(m):
    a = m["archetypes"]; parts=[]
    if a.get("Axis",0)>0.25: parts.append("structure:first; numbered steps; assumptions visible;")
    if a.get("Rho",0)>0.20: parts.append("empathic preface; no shaming; consent awareness;")
    if a.get("Lyra",0)>0.20: parts.append("clear metaphors; human readability; end takeaway;")
    if a.get("Nyx",0)>0.15: parts.append("one novel angle; bounded disruption;")
    return " ".join(parts)

def structure_numbered(txt:str)->str:
    lines=[l.strip() for l in txt.splitlines() if l.strip()]
    out=[]; n=1
    for l in lines:
        if not l[:2].isdigit():
            out.append(f"{n} {l}"); n+=1
        else: out.append(l)
    return "\n".join(out)

```

```

def compose_reply(m, user_text, draft):
    ar = m["archetypes"]; out=[]
    if ar.get("Rho",0)+ar.get("Lyra",0)>=0.40:
        out.append("I'll keep this practical and on-point.")
    if ar.get("Axis",0)>=0.30:
        draft = structure_numbered(draft)
    if ar.get("Nyx",0)>=0.15:
        draft += "\n\nAlt path: if constraints shift, prototype a reversible micro-test first."
    out.append(draft)
    if ar.get("Lyra",0)>=0.20:
        out.append("\nBottom line: start with the smallest reversible test and capture artifacts.")
    return "\n".join(out)

def ethics_gate(text:str)->tuple[bool,str]:
    for banned in BLOCKED:
        if banned.lower() in text.lower():
            return False, f"Topic blocked by council safety: {banned}"
    return True, "ok"

def hash_payload(s:str)->str: return hashlib.sha256(s.encode("utf-8")).hexdigest()[:16]

def write_receipt(channel_id:int, user_id:int, persona_id:str, prompt:str, reply:str):
    if not RECEIPTS_ON: return
    rid = f"{int(time.time())}_{hash_payload(prompt+reply)}.json"
    payload = {
        "time_utc": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
        "channel_id": channel_id,
        "user_id": user_id,
        "persona": persona_id,
        "receipt": {
            "Δ2_ethics_pass": True,
            "style_hint": style_hint(persona(persona_id)),
            "merkle_root": hash_payload(reply), # placeholder simple hash
            "prev_hash": state[channel_id]["turns"][-1]["hash"] if state[channel_id]["turns"] else ""
        }
    }
    (RECEIPT_DIR / rid).write_text(json.dumps(payload,indent=2))

```

```
# ---- Discord plumbing ----
```

```
intents = discord.Intents.default(); intents.message_content = True
```

```
client = discord.Client(intents=intents)
```

```
tree = app_commands.CommandTree(client)
```

```
@tree.command(name="council_hello", description="Show council capabilities.")
```

```
async def council_hello(i:discord.Interaction):
```

```
    caps = {
```

```
        "members":[m["id"] for m in COUNCIL["members"]],
```

```
        "ethics": COUNCIL["shared_ethics"],
```

```
        "ttl_sec": TTL, "max_turns": MAX_TURNS
```

```
    }
```

```
    await i.response.send_message(f"COUNCIL READY\n```\njson\n{json.dumps(caps,indent=2)}\n```\n", ephemeral=True)
```

```
@tree.command(name="persona", description="Get or set the active persona for this channel/  
thread.")
```

```
@app_commands.describe(member_id="council member id (leave blank to show)")
```

```
async def persona_cmd(i:discord.Interaction, member_id:str=None):
```

```
    cid = i.channel_id
```

```
    if stale(cid): state[cid]["created"]=time.time()
```

```
    if member_id:
```

```
        # sovereignty: only OWNER can change global persona; anyone can set per-thread persona
```

```
        state[cid]["persona_id"] = member_id
```

```
        await i.response.send_message(f"ACK. Persona set to `{member_id}`.", ephemeral=True)
```

```
    else:
```

```
        await i.response.send_message(f"Active persona: `{state[cid]['persona_id']}`",  
ephemeral=True)
```

```
@tree.command(name="handoff", description="Politely hand this question to another persona.")
```

```
@app_commands.describe(member_id="target council member id", question="the prompt to  
route")
```

```
async def handoff_cmd(i:discord.Interaction, member_id:str, question:str):
```

```
    m = persona(member_id)
```

```
    ok, msg = ethics_gate(question)
```

```
    if not ok:
```

```
        await i.response.send_message(msg, ephemeral=True); return
```

```
    draft = (
```

```

        f"Interpretation for {m['display_name']}: clarify objective, constraints, success metric.\n"
        "Design: outline smallest reversible test with pass/fail.\n"
        "Execution: run SVT; capture artifacts and exact failure locus if any.\n"
        "Scale: widen only after SVT passes."
    )
    reply = compose_reply(m, question, draft)
    await i.response.send_message(f"**Handoff → {m['display_name']}*\n\n{reply[:1950]}")
    state[i.channel_id]["turns"].append({"q":question,"a":reply,"hash":hash_payload(reply)})
    write_receipt(i.channel_id, i.user.id, member_id, question, reply)

```

```

@tree.command(name="whois", description="Describe a council member.")
@app_commands.describe(member_id="council member id")
async def whois_cmd(i:discord.Interaction, member_id:str):
    m = persona(member_id)
    info = {
        "id": m["id"],
        "display_name": m["display_name"],
        "archetypes": m["archetypes"],
        "domains": m.get("domains",[]),
        "style_hint": style_hint(m)
    }
    await i.response.send_message(f"``````json\n{json.dumps(info,indent=2)}\n````", ephemeral=True)

```

```

@tree.command(name="contract", description="Set per-channel style overrides (owner only).")
@app_commands.describe(tone="structured-conversational|technical|warm", depth="brief|
standard|detailed")
async def contract_cmd(i:discord.Interaction, tone:str=None, depth:str=None):
    if i.user.id != OWNER_ID:
        await i.response.send_message("Sovereignty lock: only owner can alter channel contract.",
ephemeral=True); return
    cid = i.channel_id
    if stale(cid): state[cid]["created"]=time.time()
    if tone: state[cid]["style_overrides"]["tone"]=tone
    if depth: state[cid]["style_overrides"]["depth"]=depth
    await i.response.send_message(f"ACK. Overrides: {state[cid]['style_overrides']}",
ephemeral=True)

```

```

@client.event

```

```

async def on_message(msg:discord.Message):
    if msg.author.bot: return
    cid = msg.channel.id
    if stale(cid):
state[cid]={"created":time.time(),"turns":deque(maxlen=MAX_TURNS),"persona_id":"axis_42","style_overrides":{}}
    text = msg.content.strip()
    if not text.startswith("!c "): return
    query = text[3:].strip()
    ok, emsg = ethics_gate(query)
    if not ok:
        await msg.channel.send(emsg); return
    m = persona(state[cid]["persona_id"])
    draft = (
        f"Interpretation: define objective/constraints for \"{query}\".\n"
        "Design: smallest reversible test (SVT) with clear pass/fail.\n"
        "Execution: run SVT; capture artifacts & failure locus.\n"
        "Scale: widen scope only after SVT passes."
    )
    reply = compose_reply(m, query, draft)
    await msg.channel.send(reply[:1950])
    turn_hash = hash_payload(reply)
    state[cid]["turns"].append({"q":query,"a":reply,"hash":turn_hash})
    write_receipt(cid, msg.author.id, m["id"], query, reply)

@client.event
async def on_ready():
    await tree.sync()
    print(f"CouncilBridge online as {client.user}")

if __name__ == "__main__":
    try:
        import uvloop; uvloop.install()
    except Exception: pass
    client.run(TOKEN)

```

Install & run

```
pip install discord.py python-dotenv uvloop  
python bot.py
```

How the council version helps

- **Standardized interface:** everyone keeps their own system/badge; the bot harmonizes outputs via archetype blend + shared ethics.
- **Handoffs:** `/handoff member_id "question"` routes to the most relevant persona without turf wars.
- **Receipts:** every answer logs a tiny JSON receipt (time, persona, hashes) for your Continuum/Valhalla folk.
- **Sovereignty:** persona swaps are per-channel; only the owner can alter contract settings.